

SmartCow Tracker

Sistema de Rastreo y Monitoreo de Ganado Bovino

PLAN DE DESARROLLO DEL EQUIPO

Guía de trabajo colaborativo · Sprints · Distribución de tareas · Flujo Git

Equipo	2 desarrolladores (Dev A + Dev B)
Duración estimada	16 semanas (4 meses)
Metodología	Sprints de 2 semanas (Scrum lite)
Versión	1.0.0 — Mayo 2026

1. Visión General del Proyecto

1.1 Objetivo del documento

Este documento es la guía de trabajo colaborativo del equipo de desarrollo de SmartCow Tracker. Define la distribución de responsabilidades entre los dos desarrolladores, el cronograma de sprints con tareas diarias, el flujo de trabajo en Git para evitar conflictos de merge, las reglas de comunicación y los criterios de finalización de cada sprint.

Es el documento de referencia para las reuniones de sincronización diarias (stand-up) y las revisiones de sprint cada dos semanas.

1.2 Perfiles del Equipo

Dev	Alias	Rol principal	Responsabilidades
DEV A	Juan Martín	Backend Lead	smartcow-api, smartcow-ai, prisma schema, WebSocket, MQTT, workers, infraestructura Docker.
DEV B	Juan Diego	Frontend Lead	smartcow-web, sistema de diseño, mapa GPS, componentes UI, Redux, React Query, formularios.

Regla de oro: Cada desarrollador es el dueño de su repo. Nadie hace push directo a main o develop del repo del otro sin revisión.

1.3 Estimación de Tiempo Total

Con un equipo de 2 desarrolladores trabajando en paralelo, la estimación para la versión 1.0.0 funcional y desplegada es de 16 semanas distribuidas en 8 sprints de 2 semanas cada uno.

Sprint	Semanas	Período	Objetivo principal
1	1–2	Semana 1–2	Fundamentos: DB schema, auth backend, shell frontend, setup Docker.
2	3–4	Semana 3–4	CRUD ganado completo (API + UI), importación CSV, ficha del animal.
3	5–6	Semana 5–6	GPS en tiempo real: pipeline MQTT→Redis→WebSocket→Mapa.
4	7–8	Semana 7–8	Geocercas: editor de polígonos, algoritmo Point-in-Polygon, alertas.
5	9–10	Semana 9–10	Motor de alertas completo: reglas, notificaciones email+SMS, centro de alertas UI.

6	11-12	Semana 11-12	Servicio IA Python, predicción de salud, dashboard analítico.
7	13-14	Semana 13-14	Panel admin, gestión de usuarios y roles, dispositivos IoT, reportes PDF/XLSX.
8	15-16	Semana 15-16	Modo offline, PWA, tests E2E, bug fixing, despliegue a producción.

2. Flujo de Trabajo en Git

2.1 Estrategia de Ramas

Se usa una variante simplificada de Gitflow adaptada para equipos pequeños. Cada repositorio sigue la misma estrategia de ramas:

Rama	Protegida	Descripción y reglas
main	SÍ	Solo código listo para producción. Push directo PROHIBIDO. Solo acepta PRs desde develop después de revisión del compañero.
develop	SÍ	Rama de integración. Siempre debe compilar sin errores. PRs desde feature branches. Requiere aprobación del compañero.
feature/nombre	NO	Una rama por feature o tarea. Parte de develop. Ej: feature/auth-jwt, feature/map-realtime.
fix/nombre	NO	Corrección de bugs encontrados en develop. Sigue el mismo flujo que feature.
hotfix/nombre	NO	Corrección urgente en producción. Parte de main, merge a main y develop. Solo en casos críticos.

2.2 Flujo Diario de Trabajo

Este es el flujo que cada desarrollador debe seguir todos los días de trabajo:

#	Paso	Comando / Acción
1	Actualizar develop local	git checkout develop → git pull origin develop
2	Crear rama de trabajo	git checkout -b feature/nombre-tarea
3	Trabajar y commitear	git add . → git commit -m "tipo(scope): descripción corta"
4	Push al remoto	git push origin feature/nombre-tarea
5	Abrir Pull Request	En GitHub: PR desde feature/... hacia develop. Asignar al compañero para revisión.
6	Code review	El compañero revisa, aprueba o comenta. Mínimo 1 aprobación para hacer merge.
7	Merge a develop	Merge squash (squash and merge) para mantener historial limpio.
8	Eliminar rama	git branch -d feature/nombre-tarea (local y remoto)

2.3 Convención de Commits (Conventional Commits)

Todos los commits deben seguir el estándar Conventional Commits. Esto facilita la generación automática de changelogs y hace el historial legible.

Prefijo	Cuándo usarlo	Ejemplo
feat	Nueva funcionalidad	feat(auth): implementar login con JWT y 2FA
fix	Corrección de bug	fix(gps): corregir actualización de posición en Redis
refactor	Refactorización sin cambio de comportamiento	refactor(animals): separar validación en helper
test	Agregar o modificar tests	test(alerts): agregar test unitario para HighTempRule
docs	Cambios en documentación	docs(readme): actualizar instrucciones de setup
chore	Tareas de configuración, dependencias	chore(deps): actualizar express a 4.21.1
style	Formato, espacios, punto y coma (no lógica)	style(map): aplicar prettier al componente MapView

2.4 Reglas para Evitar Conflictos

- Cada desarrollador trabaja en su propio repo. Dev A no toca smartcow-web sin avisar. Dev B no toca smartcow-api sin avisar.
- Si necesitas un cambio en el repo del otro, abre un Issue en GitHub describiendo qué necesitas y por qué. El dueño del repo lo implementa.
- Los contratos de la API (endpoints, request/response schemas) se acuerdan primero en el IDD antes de implementar. Esto elimina el 90% de los conflictos de integración.
- Nunca trabajar en la misma rama desde dos máquinas diferentes sin sincronizar primero.
- Siempre hacer git pull --rebase antes de empezar a trabajar cada día.
- Si surge un conflicto de merge, el que hizo el PR es responsable de resolverlo antes de pedir la revisión.

Regla de sincronización: Cada lunes al inicio de la semana, ambos desarrolladores sincronizan su develop local con el remoto antes de crear cualquier rama nueva.

3. Plan de Sprints Detallado

Cada sprint dura 2 semanas (10 días hábiles). La primera reunión del sprint es la Planning (lunes semana 1), la reunión de cierre es la Retrospectiva (viernes semana 2). Hay un stand-up diario de 15 minutos todos los días.

SPRINT 1 — Fundamentos y Setup · Semanas 1–2

 **Goal:** Base de datos migrada, autenticación backend funcional, shell del frontend corriendo, Docker Compose operativo.

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 1	Setup tsconfig, app.ts, logger, env config	Setup Vite, Tailwind, shadcn/ui, router	Planning Sprint 1 · Acordar contratos API de Auth
Mar	Día 2	Prisma schema: farms, users, user_farms	Layout shell: Sidebar, Header, BottomNav	—
Mié	Día 3	Prisma migrate + seed inicial · PrismaService	Páginas: Login, Dashboard esqueleto	Stand-up · Revisar schema Prisma juntos
Jue	Día 4	RedisService · Auth module: login, bcrypt	AuthContext · Formulario de login (React Hook Form + Zod)	—
Vie	Día 5	JWT RS256: generar, firmar, verificar	Integrar login con API · Token storage	Demo interna · Merge feature/auth-backend y feature/auth-frontend a develop
Lun	Día 6	Refresh token + cookie HttpOnly · Logout	Protected routes con rol · Redirect lógica	—
Mar	Día 7	RBAC middleware (authorize guard)	Dashboard con métricas placeholder	Stand-up
Mié	Día 8	2FA setup: speakeasy + QR code endpoint	Pantalla de 2FA setup en UI	—
Jue	Día 9	2FA verify endpoint · Rate limiting en /auth	Tests unitarios de AuthService	Tests del formulario Login
Vie	Día 10	Docker Compose completo (api+db+redis)	Variable VITE_API_URL · Axios client base	Revisión Sprint 1 · PR a main si todo OK · Retro

Entregables Sprint 1: Login funcional con JWT. 2FA operativo. Docker Compose levanta todos los servicios. Dev B puede autenticarse contra la API de Dev A.

SPRINT 2 — CRUD de Ganado · Semanas 3–4

 **Goal:** *Inventario completo de ganado funcional con importación CSV, ficha individual y búsqueda en tiempo real.*

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 11	Prisma schema: animals, devices · Migrar	Componentes: AnimalCard, StatusBadge	Planning Sprint 2 · Acordar contratos API Animals
Mar	Día 12	AnimalService: create, findAll, findById	CattleList: búsqueda + filtros en tiempo real	—
Mié	Día 13	AnimalService: update, soft delete · Validación Zod	CattleDetail panel lateral deslizante	Stand-up
Jue	Día 14	Importación CSV: PapaParse + batch insert	ImportModal: upload + progress + reporte errores	—
Vie	Día 15	Exportación CSV/XLSX · Tests unitarios	Ficha animal: campos técnicos, foto upload	Demo · Merge branches a develop
Lun	Día 16	Endpoint GET /animals/:id/vitals (placeholder)	VitalCard component (data estática por ahora)	—
Mar	Día 17	Endpoint GET /animals/:id/location (placeholder)	Integrar React Query en CattleList	Stand-up
Mié	Día 18	Endpoint POST /admin/devices · DeviceService	Formulario registro animal completo	—
Jue	Día 19	Tests integración CRUD animals	Tests componentes AnimalCard + CattleList	Revisión cruzada de código
Vie	Día 20	Code review + merge	Code review + merge	Revisión Sprint 2 · Demo CRUD ganado · Retro

Entregables Sprint 2: *Registrar, editar, buscar y dar de baja ganado desde la UI. Importación CSV de 100 animales en < 30s. Ficha individual completa.*

SPRINT 3 — GPS en Tiempo Real · Semanas 5–6

 **Goal:** *Mapa GPS mostrando posiciones en vivo con actualización cada 10s vía WebSocket. Pipeline MQTT completo.*

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 21	Prisma schema: gps_history, vital_history · Migrar	Setup Leaflet + React-Leaflet en MapView	Planning Sprint 3 · Acordar estructura evento WS animal:position

Mar	Día 22	MQTT.js: suscribir al broker · parsear payload IoT	MapView: render de marcadores estáticos	—
Mié	Día 23	GPSService: validar → Redis HSET → Bull MQ	AnimalMarker component con color por estado	Stand-up · Test MQTT con cliente simulado
Jue	Día 24	WebSocket Gateway: Socket.IO + Redis pub/sub	Redux slice: gpsSlice · useSelector marcadores	—
Vie	Día 25	Worker: persistir gps_history en PostgreSQL	Suscribir a animal:position · actualizar mapa	Demo integración: collar simulado → mapa se mueve
Lun	Día 26	Endpoint GET /animals/:id/history · parámetros fecha	RouteHistory: polilínea sobre el mapa	—
Mar	Día 27	Endpoint GET /animals/:id/location (Redis cache)	FilterPanel: filtrar marcadores por estado	Stand-up
Mié	Día 28	VitalHistory pipeline: sensor → DB · endpoint vitals	VitalCard: sparkline en tiempo real (WS)	—
Jue	Día 29	Tests: GPSService pipeline completo	Tests: MapView render + Redux slice	Prueba de carga: simular 50 animales
Vie	Día 30	Merge + documentar variables entorno MQTT	Merge + limpiar re-renders innecesarios	Revisión Sprint 3 · Demo mapa en vivo · Retro

Entregables Sprint 3: Mapa mostrando 50 animales con posiciones que se actualizan cada 10s. Historial de rutas de los últimos 7 días visible.

SPRINT 4 — Geocercas · Semanas 7–8

 **Goal:** Editor de geocercas poligonales en el mapa. Evaluación Point-in-Polygon en tiempo real. Alertas de cruce.

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 31	Prisma schema: geofences · Migrar	Instalar leaflet-draw · GeofenceLayer component	Planning Sprint 4 · Acordar formato polígono JSON
Mar	Día 32	GeofenceService: CRUD + cache en Redis	Editor de polígono en mapa (dibujar vértices)	—
Mié	Día 33	Algoritmo Point-in-Polygon (Ray Casting)	Visualizar geocercas activas sobre el mapa	Stand-up · Test algoritmo PiP
Jue	Día 34	Evaluación geocercas en GPSService pipeline	Modal: crear/editar geocerca (nombre, color)	—

Vie	Día 35	Generar alerta GEOFENCE_EXIT si animal sale	Notificación toast en UI al cruce de geocerca	Demo: animal cruzando geocerca genera alerta
Lun	Día 36	Invaldar cache Redis al editar geocerca	Panel gestión de geocercas (lista + toggle activo)	—
Mar	Día 37	Endpoint GET /geofences con polígonos	Cargar geocercas al iniciar MapView (React Query)	Stand-up
Mié	Día 38	Tests: algoritmo PiP (100 casos aleatorios)	Tests: GeofenceLayer render + modal	—
Jue	Día 39	Tests integración: CRUD geofences	Tests: toast notificación en cruce	Revisar cobertura de tests
Vie	Día 40	Merge + review	Merge + review	Revisión Sprint 4 · Demo geocercas · Retro

Entregables Sprint 4: Crear geocercas dibujando sobre el mapa. Alerta automática en < 30s cuando un animal cruza el límite.

SPRINT 5 — Motor de Alertas · Semanas 9–10

 **Goal:** Sistema completo de alertas: 7 reglas activas, notificaciones email+SMS, centro de alertas en UI con ciclo de vida.

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 41	Prisma schema: alerts, alert_actions · Migrar	AlertCenter: lista de alertas + filtros	Planning Sprint 5 · Acordar niveles de prioridad y eventos WS
Mar	Día 42	AlertEngine: patrón Strategy · IAlertRule	AlertItem component: borde color + acciones	—
Mié	Día 43	Implementar: HighTemperatureRule, AbnormalHeartRateRule	Redux alertsSlice · suscribir a alert:new	Stand-up
Jue	Día 44	Implementar: SensorOfflineRule · job periódico cada 5 min	Toast de alerta crítica en tiempo real	—
Vie	Día 45	Implementar: ProlongedImmobilityRule, NightMovementRule	Alert badge en campana del Header	Demo: alerta crítica dispara toast + badge
Lun	Día 46	NotificationService: SendGrid (email) · plantillas	AlertDetail panel: contexto completo	—
Mar	Día 47	NotificationService: Twilio (SMS) · rate limit Redis	Botones: Reconocer, Asignar, Cerrar alerta	Stand-up
Mié	Día 48	Endpoints: ciclo de vida alertas (ACK, assign, close)	Integrar endpoints de ciclo de vida en UI	—
Jue	Día 49	Tests: cada regla de AlertEngine por separado	Tests: AlertItem estados + acciones	Revisar flujo completo alerta → notificación
Vie	Día 50	Merge + audit_logs para acciones de alertas	Merge + animaciones del AlertCenter	Revisión Sprint 5 · Demo motor alertas · Retro

Entregables Sprint 5: 7 reglas de alerta activas. Email y SMS funcionando. Centro de alertas con reconocimiento y cierre. Tiempo de notificación < 60s.

SPRINT 6 — Servicio IA y Analytics · Semanas 11–12

 **Goal:** Microservicio Python de predicción operativo. Dashboard analítico con gráficos. Predicción de salud integrada en ficha del animal.

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
-----	-------	-----------------	------------------	-------------------

Lun	Día 51	smartcow-ai: app/main.py, config, schemas Pydantic	Dashboard analítico: estructura y layout	Planning Sprint 6 · Acordar contrato POST /predict/health
Mar	Día 52	FastAPI router /predict/health · PredictionService	LineChart actividad hato (Recharts)	—
Mié	Día 53	Modelo ML: pipeline preprocesamiento + Random Forest	Histograma distribución temperatura por raza	Stand-up · Test con datos dummy de vitales
Jue	Día 54	Serializar modelo con joblib · endpoint /health	Heatmap actividad 24h por animal	—
Vie	Día 55	AIClient en smartcow-api: HTTP + circuit breaker	Tabla top animales en riesgo (IA)	Demo: predicción de salud en ficha del animal
Lun	Día 56	Guardar predicciones en ai_predictions · cron semanal	PredictionCard en CattleDetail	—
Mar	Día 57	Endpoint GET /analytics/stats · ReportService draft	Filtros en dashboard: fecha, raza, sector	Stand-up
Mié	Día 58	Exportación XLSX de reportes · Bull MQ job	Botón exportar reporte + descarga automática	—
Jue	Día 59	Tests: PredictionService con datos mock	Tests: gráficos Recharts render	Simular 90 días datos para entrenar modelo
Vie	Día 60	Merge + Dockerfile smartcow-ai	Merge + loading states analítica	Revisión Sprint 6 · Demo IA + analytics · Retro

Entregables Sprint 6: Predicción LOW/MEDIUM/HIGH por animal funcionando. Dashboard analítico con 4 gráficos. Exportación XLSX de reportes.

SPRINT 7 — Panel Admin y Reportes · Semanas 13–14

 **Goal:** Gestión completa de usuarios, roles, dispositivos IoT, geocercas desde panel admin. Reportes PDF.

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 61	Endpoints admin: CRUD users, roles, invitar usuarios	Pantalla admin/users: tabla + formulario	Planning Sprint 7
Mar	Día 62	Endpoints admin: CRUD devices, asignar a animal	Pantalla admin/devices: tabla + estados	—
Mié	Día 63	FarmThresholds: configurar umbrales por finca	Pantalla settings: formulario umbrales	Stand-up

Jue	Día 64	Generación PDF reportes (pdfkit) · Bull MQ job	Vista previa de reporte en modal	—
Vie	Día 65	Exportar historial GPS de un animal	Descargar PDF/XLSX desde la ficha del animal	Demo: admin gestiona usuarios y descarga reporte
Lun	Día 66	health_events: registrar vacunas y tratamientos	HealthTimeline en CattleDetail	—
Mar	Día 67	Audit logs: registrar acciones críticas	Pantalla audit trail para Super Admin	Stand-up
Mié	Día 68	RBAC: verificar permisos en todos los endpoints	Ocultar/mostrar UI según rol del usuario	—
Jue	Día 69	Tests integración endpoints admin	Tests: formularios admin + RBAC en UI	Revisar cobertura global de tests
Vie	Día 70	Merge + review	Merge + review	Revisión Sprint 7 · Demo panel admin · Retro

Entregables Sprint 7: Panel admin completo con usuarios, roles, dispositivos. Reportes PDF y XLSX descargables. Historial médico del animal.

SPRINT 8 — Pulido, Offline y Deploy · Semanas 15–16

 **Goal:** Modo offline con Service Worker. Tests E2E. Bug fixing general. Despliegue a producción en Railway + Vercel.

Día	Fecha	Dev A — Backend	Dev B — Frontend	Sync / Compartido
Lun	Día 71	Dockerfiles finales: multi-stage builds	Service Worker: Workbox setup + offline shell	Planning Sprint 8 · Lista de bugs conocidos
Mar	Día 72	docker-compose.prod.yml · variables entorno prod	Offline: cache API responses en IndexedDB	—
Mié	Día 73	Setup Railway: deploy smartcow-api + smartcow-ai	Manifest PWA · iconos · instalable en Android	Stand-up
Jue	Día 74	PostgreSQL en Supabase: migrar schema + seed	Tests E2E Playwright: flujo login → dashboard	—
Vie	Día 75	Redis Upstash · Mosquitto en Railway	Tests E2E: localizar animal → reconocer alerta	Demo producción: todo funcionando en Railway
Lun	Día 76	Nginx config: proxy + gzip + headers seguridad	Lighthouse audit: performance, accesibilidad	—
Mar	Día 77	Monitoreo: health checks · logs en producción	Bug fixing: issues encontrados en prod	Stand-up

Mié	Día 78	Backup PostgreSQL automático · script	Bug fixing frontend + optimización bundle	—
Jue	Día 79	Documentar variables entorno en .env.example	README final: instrucciones deploy paso a paso	Revisión cruzada completa del sistema
Vie	Día 80	Deploy final + tag v1.0.0 en todos los repos	Deploy final + tag v1.0.0	🎉 Revisión Sprint 8 · Demo v1.0.0 · Celebración

Entregables Sprint 8: Aplicación desplegada y accesible en producción. PWA instalable. Modo offline funcional. Tag v1.0.0 en todos los repos.

4. Protocolo de Reuniones

4.1 Stand-up Diario (15 minutos)

Todos los días hábiles, al inicio de la jornada. Puede ser presencial, por videollamada o por chat (WhatsApp/Telegram). Cada dev responde exactamente 3 preguntas:

#	Pregunta	Ejemplo de respuesta
1	¿Qué hice ayer?	"Terminé el AuthService con JWT y empujé el PR #12 a develop."
2	¿Qué voy a hacer hoy?	"Voy a implementar el refresh token y el middleware RBAC."
3	¿Hay algún bloqueante?	"Necesito que Dev B me confirme el formato del evento WS para el mapa."

4.2 Planning de Sprint (Lunes, inicio de sprint)

- Duración: 1-2 horas.
- Revisar las tareas del sprint que empieza según este documento.
- Ajustar tareas si algo cambió o se atrasó en el sprint anterior.
- Acordar contratos de API y eventos WebSocket que se necesiten en el sprint.
- Asignar tareas en GitHub Projects (columnas: To Do → In Progress → In Review → Done).

4.3 Revisión de Sprint (Viernes, fin de sprint)

- Duración: 1 hora.
- Demo de lo construido: cada dev muestra lo que terminó.
- Verificar que los entregables del sprint están completos.
- Merge de las ramas de develop a main si el sprint está completo.
- Tag en git: `git tag -a sprint-N -m "Sprint N completado"`

4.4 Retrospectiva (Viernes, al final de la revisión)

- ¿Qué salió bien en este sprint?
- ¿Qué salió mal o fue más difícil de lo esperado?
- ¿Qué cambiamos para el próximo sprint?
- Máximo 30 minutos. Una acción concreta de mejora por sprint.

5. Reglas de Integración entre Repos

5.1 Contrato primero, código después

La causa número uno de conflictos de integración es que Dev A implementa un endpoint de una forma y Dev B lo consume de otra. Para prevenirlo, la regla es simple: antes de implementar cualquier funcionalidad que involucre comunicación entre repos, se acuerda el contrato.

¿Qué acordar antes?	Cómo acordarlo
Nuevo endpoint REST	Dev A describe en un comentario de GitHub Issue: método HTTP, URL, request body (ejemplo JSON), response body (ejemplo JSON), código HTTP de éxito y errores. Dev B aprueba antes de implementar.
Nuevo evento WebSocket	Dev A describe: nombre del evento, payload JSON exacto con tipos. Dev B confirma que puede consumirlo. Se documenta en el IDD.
Cambio en schema DB	Dev A informa el cambio con al menos 1 día de antelación. Dev B verifica que ningún tipo de TypeScript generado por Prisma rompe su código.
Nuevo campo en respuesta	Agregar campos es siempre retrocompatible (no rompe). Eliminar o renombrar campos SIEMPRE requiere coordinación con Dev B antes de hacer el merge.

5.2 GitHub Issues como canal de comunicación técnica

Todo cambio que afecte a ambos repos debe tener un Issue en GitHub antes de implementarse. El flujo es:

#	Paso	Descripción
1	Crear Issue	El que necesita el cambio crea un Issue en el repo del otro con el label "contract" describiendo qué necesita y por qué.
2	Discutir	Ambos comentan en el Issue hasta llegar a un acuerdo. Nadie empieza a codear hasta que el Issue tenga el label "approved".
3	Implementar	El dueño del repo implementa el cambio acordado. El que lo pidió puede probar contra develop.
4	Cerrar Issue	El Issue se cierra cuando ambos han verificado que la integración funciona correctamente.

5.3 Variables de Entorno y Secretos

- Nunca commitear archivos .env al repositorio. Solo .env.example con valores placeholder.
- Los secretos de producción (API keys, JWT private) se comparten entre los dos devs por un canal seguro (Signal, 1Password, etc.), nunca por WhatsApp o email.
- Cada vez que se agrega una variable de entorno nueva, se actualiza inmediatamente .env.example y se avisa al compañero.

5.4 Checklist antes de abrir un PR

- El código compila sin errores (npm run build o equivalente).
- Los tests pasan (npm test o pytest).
- No hay console.log de debug en el código.
- El .env.example está actualizado si se agregaron variables.
- La descripción del PR explica QUÉ hace el cambio y POR QUÉ.
- Si toca un contrato de API: el compañero ya fue avisado y aprobó.

6. Goals del Proyecto

6.1 Goals por Sprint

Sprint	Fecha objetivo	Definition of Done
S1	Fin semana 2	Login funcional. JWT + 2FA. Docker Compose levanta. Frontend muestra pantalla de login.
S2	Fin semana 4	CRUD ganado completo. Importación CSV. Ficha individual. Búsqueda en tiempo real.
S3	Fin semana 6	Mapa con 50+ animales actualizándose cada 10s. Historial de rutas visible.
S4	Fin semana 8	Geocercas creadas en el mapa. Alerta en < 30s al cruzar límite.
S5	Fin semana 10	7 reglas de alerta activas. Email + SMS funcionando. Centro de alertas con ciclo de vida.
S6	Fin semana 12	Predicción de salud por animal. Dashboard analítico con gráficos. Exportación XLSX.
S7	Fin semana 14	Panel admin completo. Reportes PDF. Historial médico. Audit logs.
S8	Fin semana 16	Aplicación en producción. PWA instalable. Modo offline. Tag v1.0.0.

6.2 Goals de Calidad (No Negociables)

- Cobertura de tests $\geq 70\%$ en módulos críticos (auth, alerts, gps).
- Latencia GPS extremo a extremo ≤ 2 segundos en condiciones normales.
- Tiempo de carga inicial del dashboard < 3 segundos en 10 Mbps.
- Disponibilidad en producción $\geq 99\%$ durante las primeras 4 semanas post-lanzamiento.
- Cero vulnerabilidades HIGH en el scan de dependencias (npm audit + pip check).

6.3 Goal Final — v1.0.0

Definición de Hecho v1.0.0: *Un ganadero puede registrar su finca, agregar su ganado, colocar los collares IoT, definir geocercas, recibir alertas en tiempo real por web/email/SMS, ver el mapa GPS en vivo y obtener predicciones de salud por animal. Todo corriendo en producción sin intervención técnica del equipo.*

— SmartCow Tracker · Plan de Desarrollo del Equipo v1.0.0 —
16 semanas · 8 sprints · 80 días hábiles · 2 desarrolladores · Mayo – Septiembre 2026